

NIFTY100 Analytics

Analyst Guide

User manual for the Streamlit dashboard, FastAPI backend, financial analytics, reporting and testing workflow.

Project Version: 1.0.0

Technology: Python, SQLite, FastAPI, Streamlit, Plotly

Coverage: NIFTY100 financial and analytical data

1. Project Overview

NIFTY100 Analytics is an analytics platform designed to provide structured financial analysis of companies in the NIFTY100 universe. The application combines an SQLite database, Python analytics modules, a FastAPI backend and a Streamlit dashboard.

The system provides company-level analysis, stock screening, peer comparison, trend analysis, sector analysis, capital allocation visualisation and annual-report access.

Primary components

- ETL pipeline for loading and validating source data.
- SQLite database containing the processed financial datasets.
- Analytics modules for ratios, CAGR, peer analysis, clustering and valuation.
- FastAPI REST API exposing analytical functionality.
- Streamlit dashboard providing the analyst-facing interface.
- PDF tearsheet and reporting functionality.
- Automated API and project tests.

Technology stack

Component	Technology
Programming language	Python
Database	SQLite
API	FastAPI
Dashboard	Streamlit
Charts	Plotly
Testing	pytest
Formatting	Black
Linting	Ruff

2. System Architecture

The application follows a layered architecture. Source data is processed by the ETL layer and stored in SQLite. Analytics modules operate on the database data. FastAPI exposes backend functionality, while Streamlit consumes the analytical functionality through the dashboard interface.

Major layers

- ETL layer — data loading, normalisation, validation and database loading.
- Analytics layer — financial ratios, CAGR, peer analysis, clustering, radar analysis and valuation.
- API layer — FastAPI routers for companies, screener, sectors, peers, valuation, portfolio, documents and health.
- Dashboard layer — Streamlit pages for analyst interaction.
- Reporting layer — portfolio summaries, sector reports and company tearsheets.

Database

The main SQLite database is located at:

```
data/database/nifty100.db
```

During Day 43 performance optimisation, indexes were added to `company_id` and `year` columns in the major financial tables. The final test suite completed with 139 passing tests.

3. Setup and Application Startup

Install project dependencies

Create or activate the project virtual environment and install the required Python dependencies.

The project uses pytest configuration from pyproject.toml. The repository root is included in the Python path, allowing imports such as src.api.main.

Start the Streamlit dashboard

Run the following command from the project root:

```
streamlit run src/dashboard/app.py
```

Start FastAPI

Run the API using:

```
uvicorn src.api.main:app --reload --port 8000
```

The API application object is defined as src.api.main:app.

Running both applications

FastAPI and Streamlit use different ports, allowing both applications to run simultaneously. FastAPI uses port 8000 while Streamlit uses port 8501 by default.

4. Dashboard Navigation

The Streamlit application provides a sidebar navigation menu. The current dashboard implementation exposes the following screens:

Navigation	Module	Purpose
HOME	01_home	Application overview and quick navigation.
COMPANY PROFILE	02_profile	Company-level financial information and charts.
SCREENER	03_screener	Filter companies using financial metrics.
PEER COMPARISON	04_peers	Compare companies against peers.
TREND ANALYSIS	05_trends	Analyse historical financial trends.
SECTOR ANALYSIS	06_sectors	Analyse sector-level financial characteristics.
CAPITAL ALLOCATION	07_capital	Visualise company capital allocation patterns.
ANNUAL REPORTS	08_reports	Access available annual reports.

The dashboard application uses a page mapping in `src/dashboard/app.py` and dynamically imports the selected page.

5. Company Profile

The Company Profile screen is intended for detailed analysis of an individual company. It provides company information, key financial metrics, historical performance and interactive charts.

Recommended workflow

- Open COMPANY PROFILE from the sidebar.
- Select the company that you want to analyse.
- Review the headline financial metrics.
- Review historical financial performance.
- Use the interactive charts to identify trends.
- Use the information together with peer and sector analysis for context.

Analyst considerations

Financial values may be unavailable for some companies or periods. The dashboard handles missing values by displaying N/A rather than allowing missing values to crash the application.

Companies with shorter financial histories should not automatically be treated as having poor performance. The available historical period should be considered when interpreting trends and CAGR values.

6. Stock Screener

The Stock Screener is designed to narrow the NIFTY100 universe using financial criteria. Filters can be combined to create a focused candidate set.

Available analytical filters

- Minimum ROE — selects companies with return on equity above the selected threshold.
- Maximum Debt-to-Equity — limits financial leverage.
- Minimum Free Cash Flow — identifies companies above a specified FCF threshold.
- Sector — restricts results to a selected sector.
- Minimum 5-year Revenue CAGR — filters based on historical revenue growth.
- Minimum 5-year PAT CAGR — filters based on historical profit growth.
- Maximum P/E — valuation filtering is subject to the availability of P/E data in the backend.

Example API screener call

```
curl "http://127.0.0.1:8000/api/v1/screener?min_roe=15"
```

The API returns a result set containing the companies that satisfy the selected criteria.

7. Peer, Trend and Sector Analysis

Peer Comparison

Peer Comparison allows an analyst to evaluate a company against comparable companies. This is useful for identifying relative strengths and weaknesses in profitability, leverage and other financial characteristics.

Trend Analysis

Trend Analysis displays historical financial trends using interactive line charts. Multiple metrics can be compared to understand whether financial performance is improving, deteriorating or remaining stable.

Sector Analysis

Sector Analysis provides sector-level comparisons. The dashboard uses interactive visualisations and sector median KPI comparisons to show differences between sectors.

The current database contains 10 populated broad-sector categories. The API therefore returns 10 sector records. Analysts should treat the database contents as the source of truth rather than assuming a fixed number of sectors.

8. Capital Allocation and Annual Reports

Capital Allocation

The Capital Allocation screen visualises companies according to capital allocation patterns. The project documentation describes an interactive treemap as the primary visualisation.

Use the screen to identify groups of companies with similar capital allocation characteristics and then investigate individual companies using the Company Profile screen.

Annual Reports

The Annual Reports screen provides access to available annual-report links. Report availability can vary by company and reporting period.

Unavailable report links are handled gracefully by the dashboard. If a report is unavailable, use the available company information and other analytical screens instead.

9. Generating PDF Tearsheets

Company tearsheets provide a compact report for an individual company. The API exposes a tearsheet endpoint under the company resource.

API endpoint

```
GET /api/v1/companies/{ticker}/tearsheet
```

Example using curl

```
curl -o TCS_tearsheet.pdf "http://127.0.0.1:8000/api/v1/companies/TCS/tearsheet"
```

The endpoint returns the PDF file when the tearsheet exists. If the company exists but the tearsheet file is not available, the API returns a 404 response.

Recommended analyst workflow

- Confirm that FastAPI is running.
- Identify the company ticker.
- Request the tearsheet endpoint.
- Save the returned PDF.
- Review the tearsheet alongside the dashboard analysis.

10. FastAPI Usage

The backend exposes REST endpoints under `/api/v1`. FastAPI's interactive documentation can be opened at `/docs` while the server is running.

Endpoint	Purpose
<code>/api/v1/health</code>	Health and database status.
<code>/api/v1/companies</code>	Company listing and company analysis.
<code>/api/v1/screener</code>	Financial screening.
<code>/api/v1/sectors</code>	Sector analysis.
<code>/api/v1/peers</code>	Peer comparison.
<code>/api/v1/valuation</code>	Valuation analysis.
<code>/api/v1/portfolio</code>	Portfolio analysis.
<code>/api/v1/documents</code>	Document-related functionality.

11. API curl Examples

Health check

```
curl "http://127.0.0.1:8000/api/v1/health"
```

Get companies

```
curl "http://127.0.0.1:8000/api/v1/companies"
```

Get TCS

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS"
```

Get sectors

```
curl "http://127.0.0.1:8000/api/v1/sectors"
```

Get screener results

```
curl "http://127.0.0.1:8000/api/v1/screener?min_roe=15"
```

Get TCS profit and loss

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/pl"
```

Get TCS balance sheet

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/bs"
```

Get TCS cash flow

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/cashflow"
```

Get TCS ratios

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/ratios"
```

12. ETL and Data Workflow

The ETL portion of the project is responsible for preparing source data for analysis. The `src/etl` package contains the loader, normaliser, validator, SQLite loader and database verification components.

Important ETL modules

- `src/etl/loader.py` — primary loading workflow.
- `src/etl/normaliser.py` — normalisation utilities.
- `src/etl/validator.py` — data-quality validation.
- `src/etl/sqlite_loader.py` — SQLite loading functionality.
- `src/etl/db_verify.py` — database verification.
- `src/etl/load_audit.py` — loading/audit support.
- `src/etl/manual_review.py` — manual review workflow.

Run ETL commands according to the project's existing scripts and configuration. Always verify the SQLite database after loading and before running analytical or dashboard components.

13. Testing

The project uses pytest. The pytest configuration is stored in `pyproject.toml` and specifies the repository root as the Python path and `tests/` as the test directory.

Run the complete test suite

```
python -m pytest tests/
```

The Day 43 final verification produced 139 passing tests with no reported failures.

Performance testing

The performance test sent 10 concurrent screener requests using Python threading. All 10 requests completed in approximately 0.234 seconds, which is comfortably below the 10-second target.

14. Troubleshooting

ModuleNotFoundError: No module named 'src'

Run commands from the project root and use the configured pytest Python path. Avoid launching test files directly with python when the project expects package imports.

API returns 404

Confirm FastAPI is running and verify that the requested router is included in `src/api/main.py`. Check `/docs` to see the registered routes.

Streamlit page does not load

Confirm Streamlit was started using `streamlit run src/dashboard/app.py` and check the terminal for import or runtime errors.

Database error

Verify that `data/database/nifty100.db` exists and that the application is using the expected database path.

Missing financial value

The dashboard may display N/A when a financial field is unavailable. This is expected behavior for missing data.

Annual report unavailable

Report availability depends on the source data and available links. Use the dashboard's available links or another company analysis screen.

Tearsheet returns 404

Confirm the ticker exists and that the corresponding PDF exists in the `reports/tearsheets` directory.

15. Performance and Optimisation

Performance testing was completed as part of Day 43. The API successfully handled the required concurrent screener workload.

Observed result

- 10 concurrent screener requests completed in approximately 0.234 seconds.
- Target: all 10 requests within 10 seconds.
- Result: PASS.
- Company Profile performance was tested against the required sub-3-second target.
- SQLite indexes were added to `company_id` and `year` columns in large financial tables.
- Final test suite result: 139 passed.

Indexes added

- `profit_loss.company_id`
- `profit_loss.year`
- `balance_sheet.company_id`
- `balance_sheet.year`
- `cash_flow.company_id`
- `cash_flow.year`
- `financial_ratios.company_id`
- `financial_ratios.year`
- `documents.company_id`
- `documents.year`

16. Project Directory Reference

```
nifty100-analytics/  
■■■ src/  
■ ■■■ analytics/  
■ ■■■ api/  
■ ■ ■■■ main.py  
■ ■ ■■■ routers/  
■ ■■■ dashboard/  
■ ■ ■■■ app.py  
■ ■ ■■■ pages/  
■ ■ ■■■ utils/  
■ ■■■ etl/  
■ ■■■ nlp/  
■ ■■■ reports/  
■ ■■■ screener/  
■■■ data/  
■ ■■■ database/  
■ ■■■ nifty100.db  
■■■ docs/  
■■■ output/  
■■■ reports/  
■■■ tests/  
■■■ pyproject.toml  
■■■ README.md
```

Key source locations

- API entry point: src/api/main.py
- Dashboard entry point: src/dashboard/app.py
- API routers: src/api/routers/
- Dashboard pages: src/dashboard/pages/
- ETL modules: src/etl/
- Analytics modules: src/analytics/
- Reporting modules: src/reports/
- Screener modules: src/screener/
- Tests: tests/
- SQLite database: data/database/nifty100.db

17. Analyst Quick Reference

Start dashboard

```
streamlit run src/dashboard/app.py
```

Start API

```
uvicorn src.api.main:app --reload --port 8000
```

API documentation

```
http://127.0.0.1:8000/docs
```

Run tests

```
python -m pytest tests/
```

Generate a TCS tearsheet

```
curl -o TCS_tearsheet.pdf "http://127.0.0.1:8000/api/v1/companies/TCS/tearsheet"
```

Final Day 43 status: performance targets passed, SQLite optimisation completed and 139 tests passed.